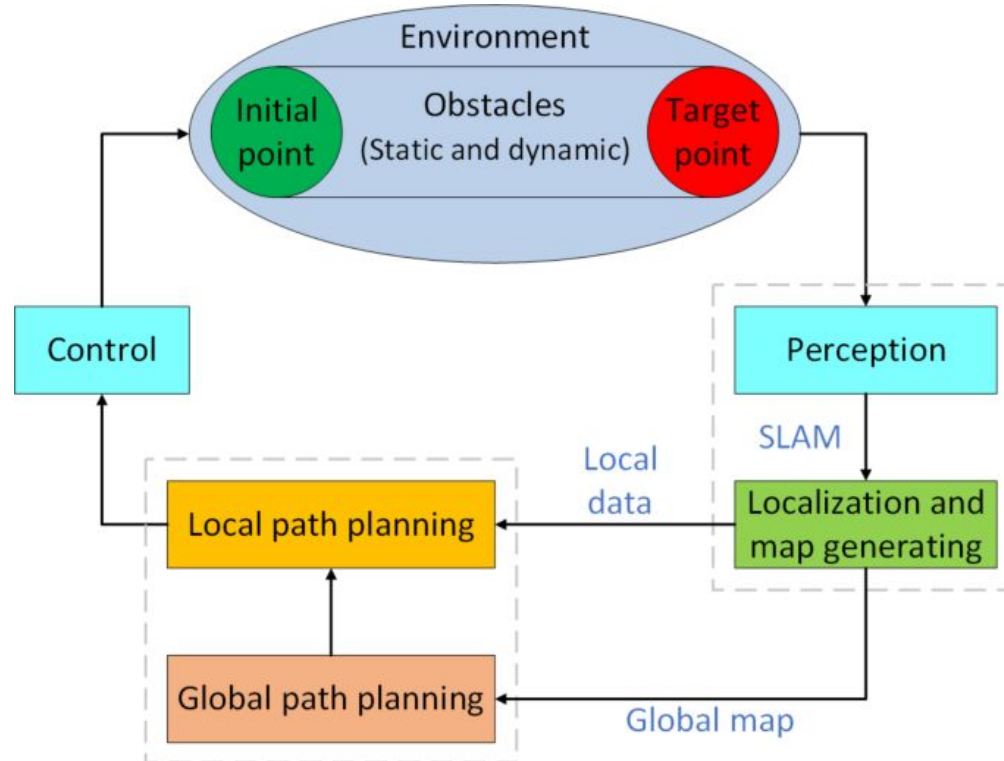


Motion Planning 1

Configuration Spaces & Graph Methods

The Motion Planning Problem



Local vs Global Planners

1. Global Path Planning

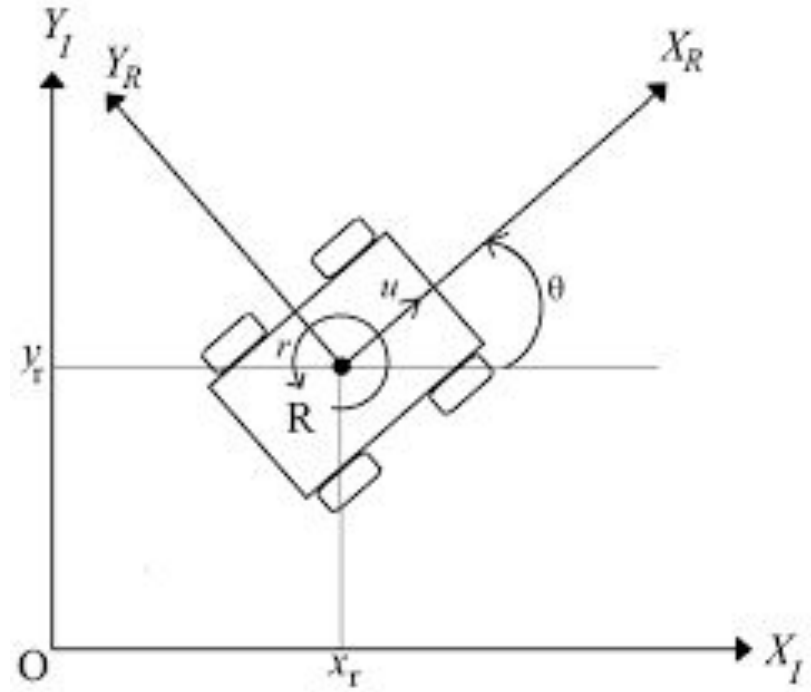


2. Local Path Planning



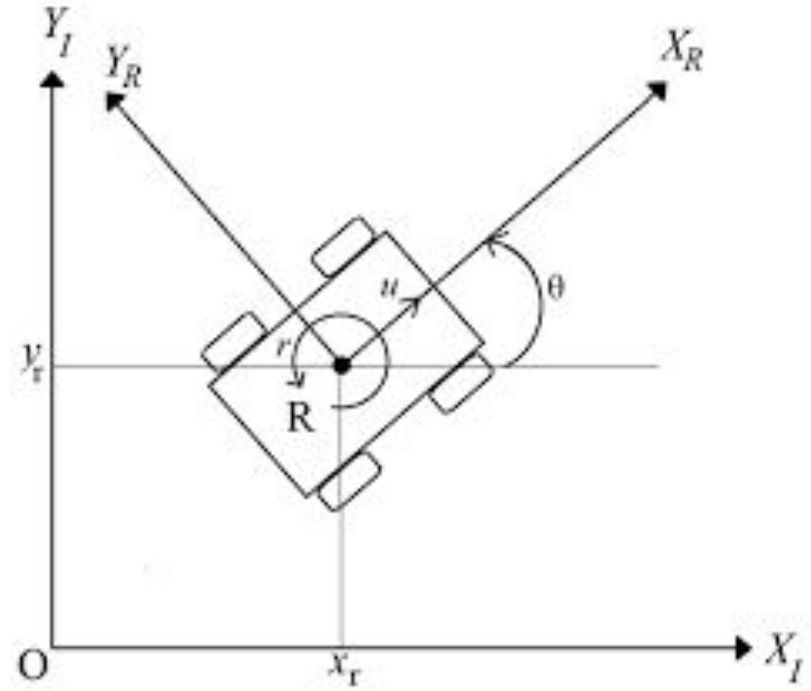
Robot Representation

- Every atom?



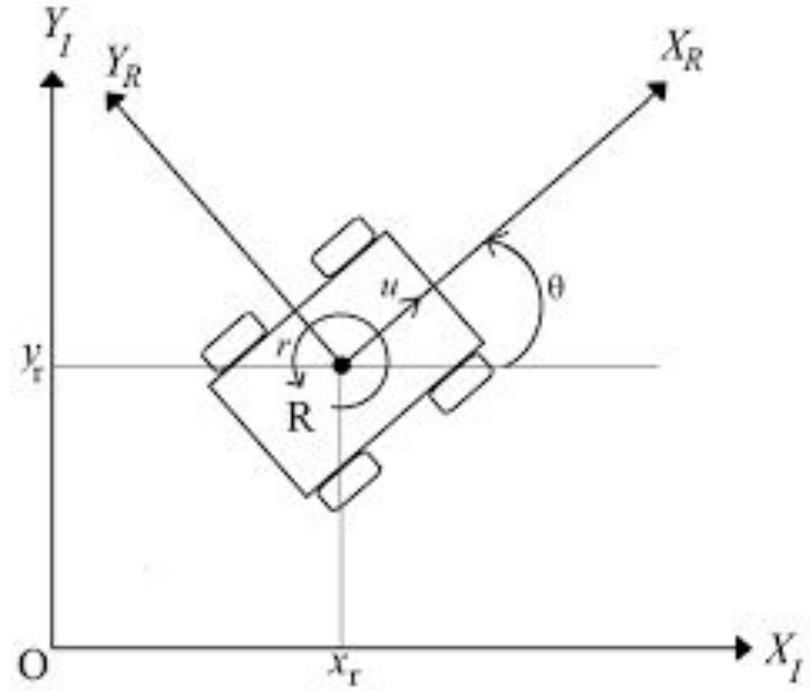
Robot Representation

- ~~Every atom?~~
- One object frame with every part having fixed dimensions and a relative transform to it?



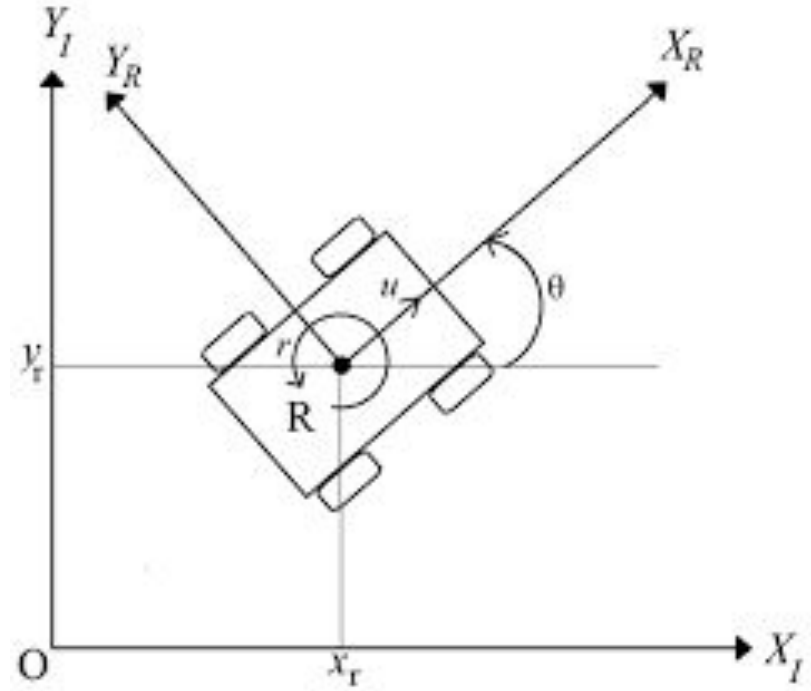
Robot Representation

- ~~— Every atom?~~
- ~~— One object frame with every part having fixed dimensions and a relative transform to it?~~
- An oriented bounding box?



Robot Representation

- ~~— Every atom?~~
- ~~— One object frame with every part having fixed dimensions and a relative transform to it?~~
- ~~— An oriented bounding box?~~
- One point in C-space



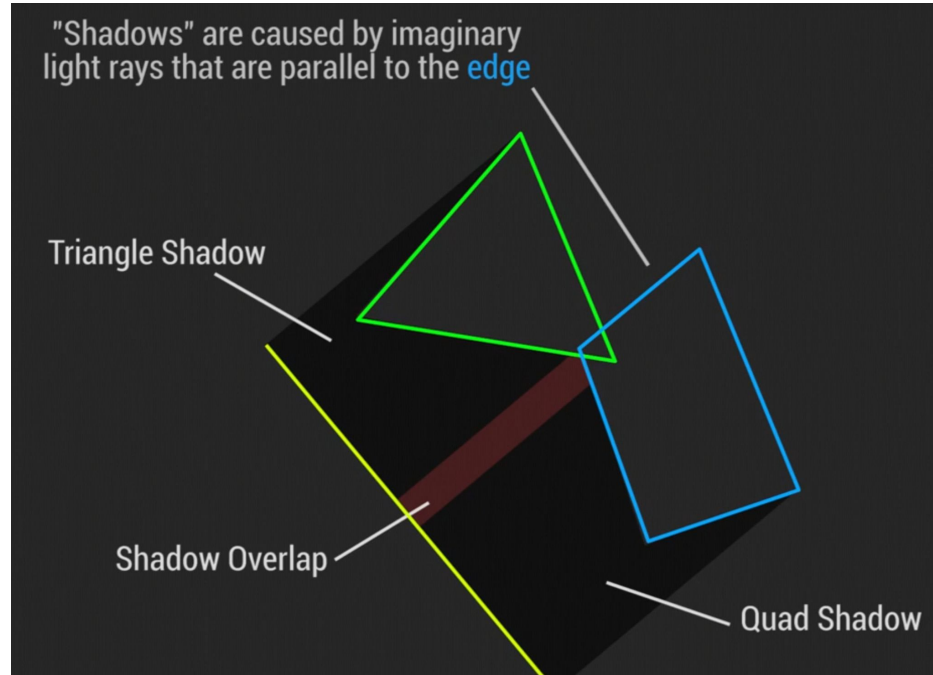
Configuration Space

- Configuration of a robot is a specification of the position of every point in the robot
- A configuration $\mathbf{q}$ completely specifies the position of every point in the robot
- Configuration space (C-space) – space of all possible configurations $\mathbf{q}$ of the robot
- Degrees of Freedom - minimum number of parameters required to specify the configuration

Robot as a Point Mass

- Planning can be done in a MUCH simpler space!

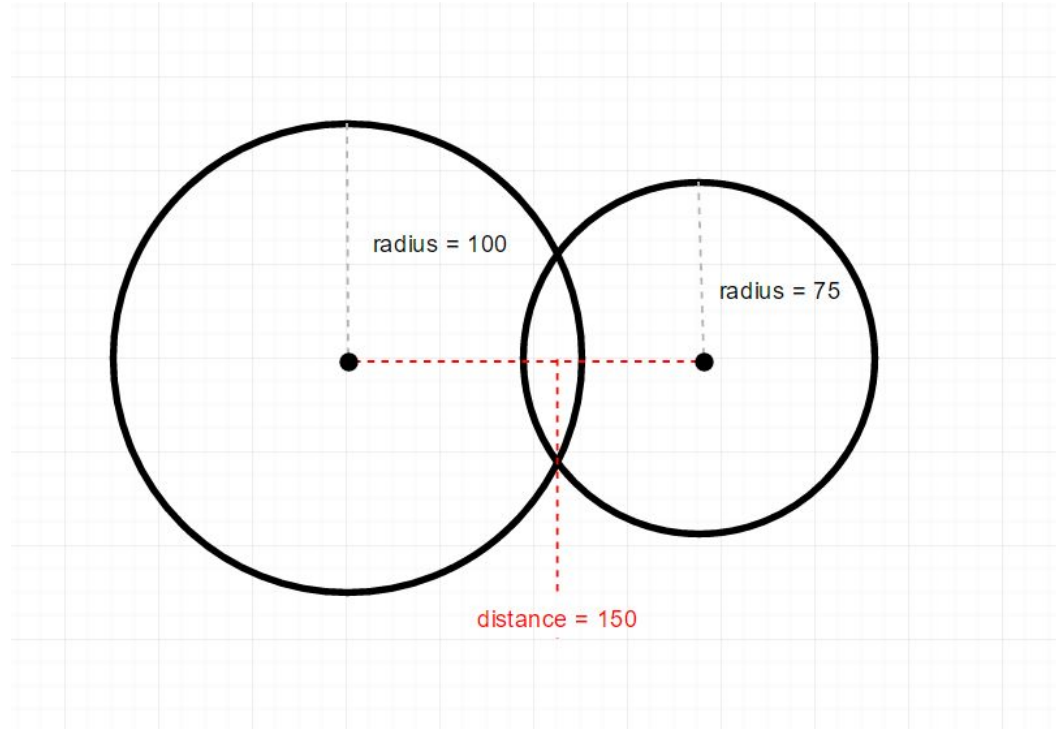
But what about Collisions?



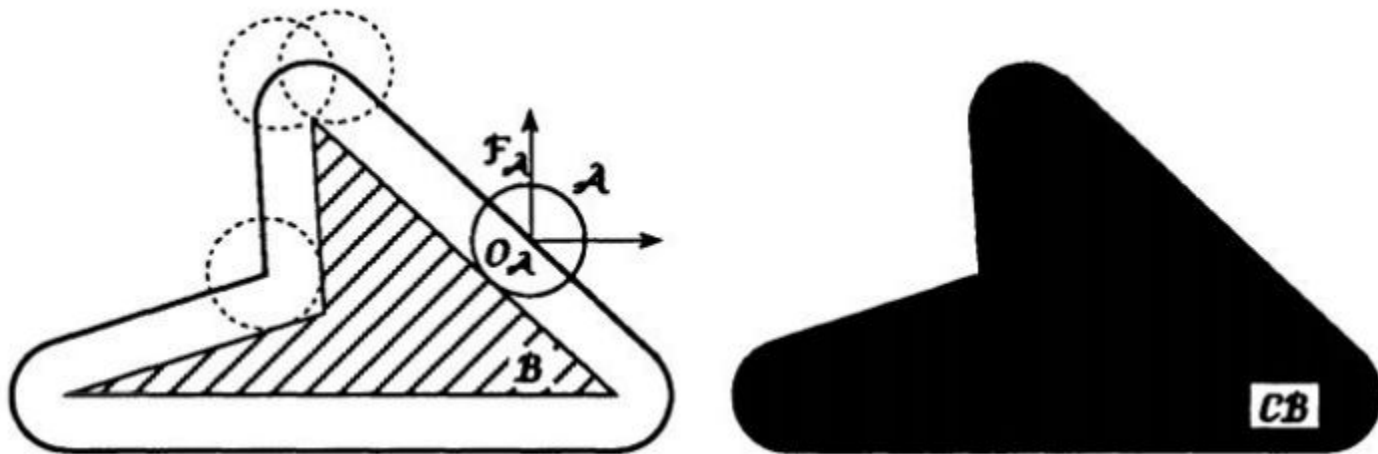
Simplest possible check?

Can you do the same check for a point robot model?

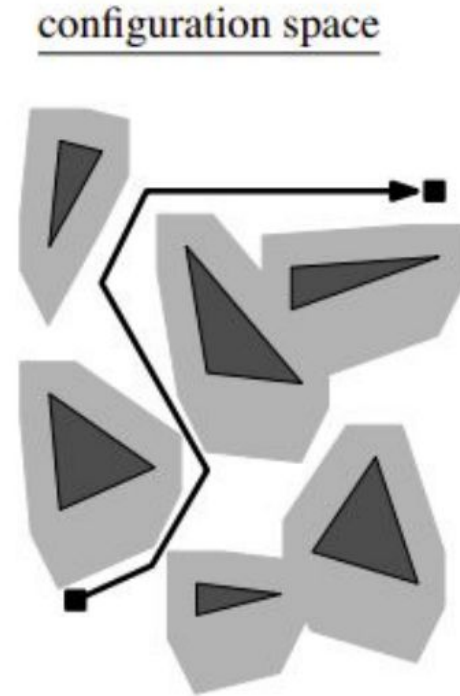
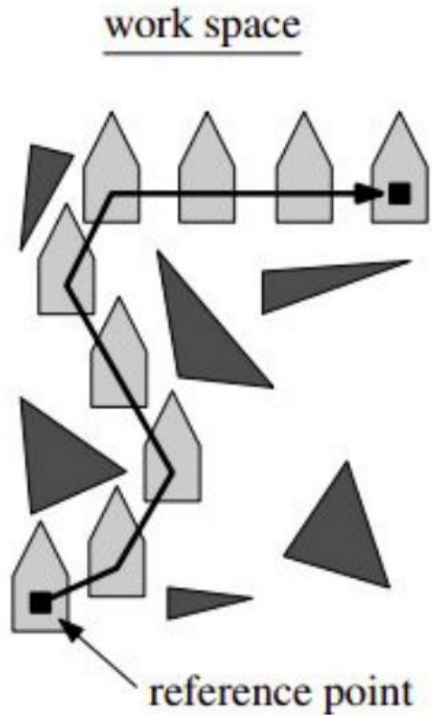
$$r_{\text{obs_new}} = r_{\text{robot}} + r_{\text{obs}}$$



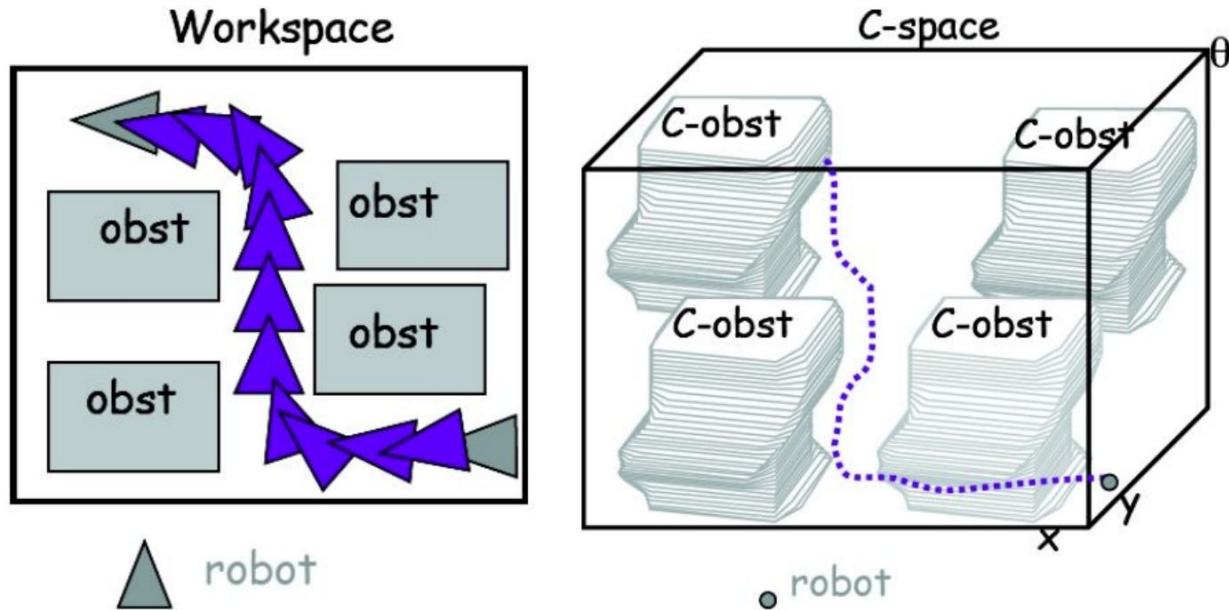
What if the obstacle is not a circle?



What if the robot is not a circle either?



Configuration Space Planning



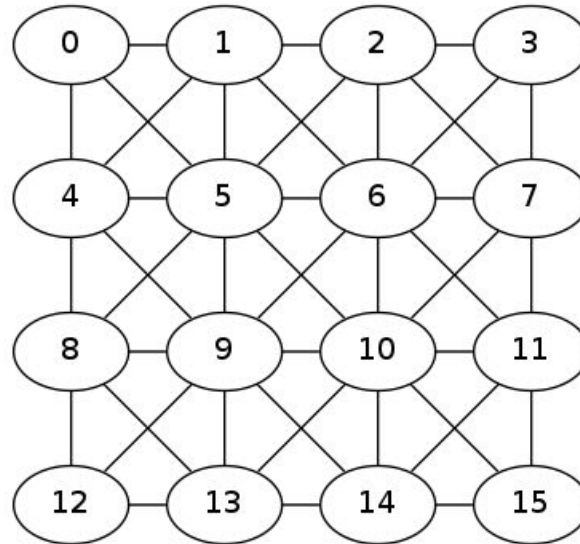
[Fig from Jyh-Ming Lien]

Path vs Trajectory

- A **path** describes the **geometric route** from start to goal.
- A **trajectory** describes the robot's **motion over time**.

Path Planning in C-Space

- C-space is continuous.
- So we discretize it and make each **non occupied cell** into a node of a graph!



Dijkstra

- Finds the shortest path from one source node to all other nodes in a weighted graph (weights here are the distances)
- Repeatedly picks the nearest unvisited node and updates distances to its neighbors

dijkstra(v):

 pq = new PriorityQueue()

 pq.insert([dest:v, pred:null, cost:0])

 while (!pq.isEmpty()):

 [dest, pred, cost] = pq.removeMin()

 if dest is unvisited:

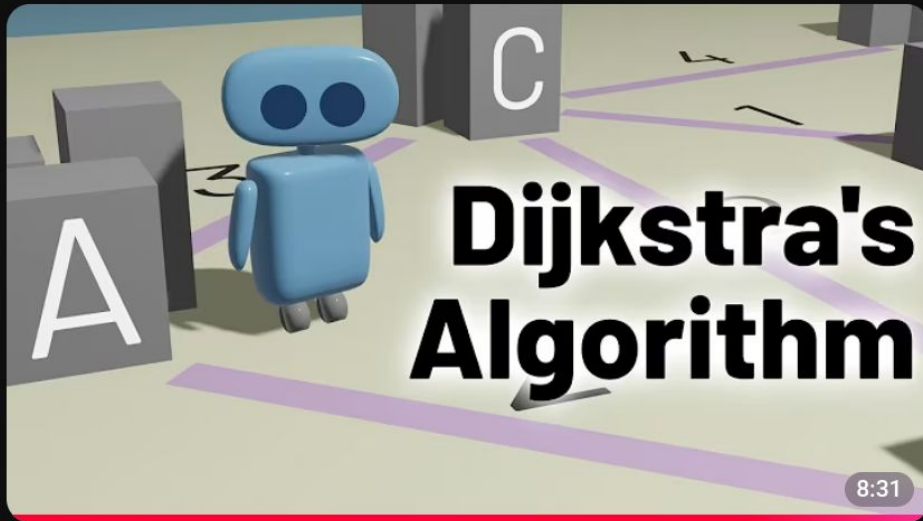
 mark dest as visited, store pred and cost for dest

 for each edge with weight w to unvisited neighbor u of dest:

 pq.insert([u, dest, cost + w])

Dijkstra

https://youtu.be/EFg3u_E6eHU



How Dijkstra's Algorithm Works

2M • 5y ago



Spanning Tree

Dijkstra's Algorithm allows us to find the shortest path between two vertices in a graph...

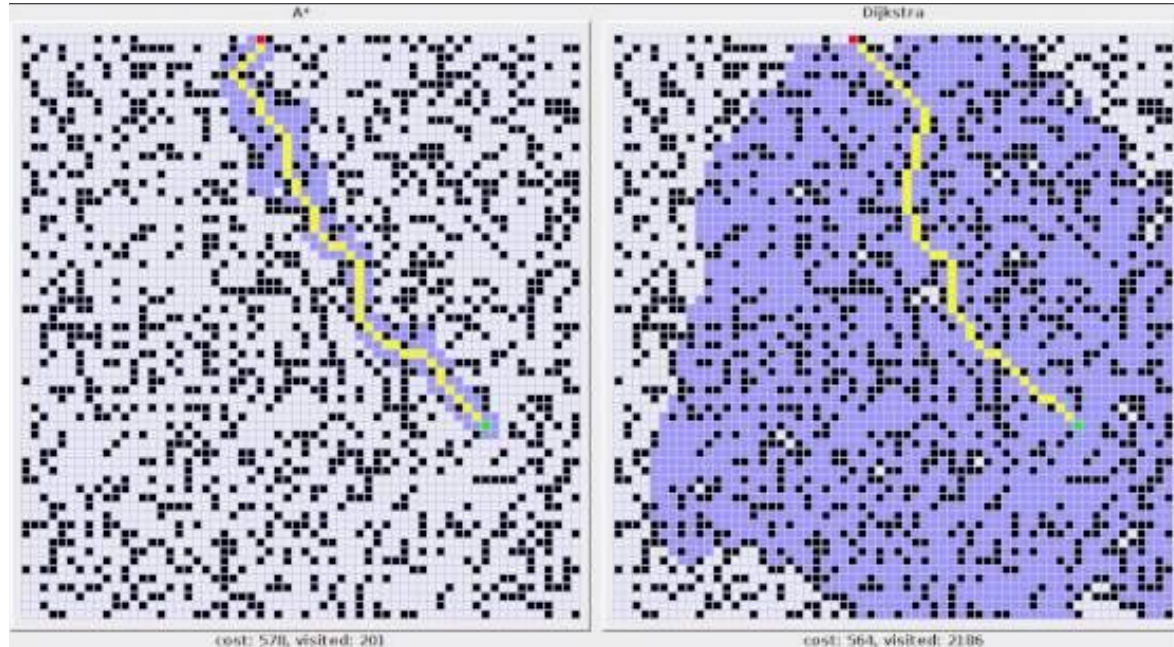


7 chapters Introduction | Finding the shortest path | Updating...



A*

- Use a heuristic in the cost to search efficiently
- Cost = cost from start + heuristic



A*

Admissible heuristic - never overestimates the actual cost to reach the goal from any node

Manhattan distance

```
function heuristic(node) =  
    dx = abs(node.x - goal.x)  
    dy = abs(node.y - goal.y)  
    return D * (dx + dy)
```

Diagonal distance

```
function heuristic(node) =  
    dx = abs(node.x - goal.x)  
    dy = abs(node.y - goal.y)  
    return D * (dx + dy) + (D2 - 2 * D) * min(dx, dy)
```

Euclidean distance

```
function heuristic(node) =  
    dx = abs(node.x - goal.x)  
    dy = abs(node.y - goal.y)  
    return D * sqrt(dx * dx + dy * dy)
```

Curse of Dimensionality

- What if q was not just 2d or 3d?
- Number of neighbours in 2d -> 8
- Number of neighbours in 3d -> 26
- Number of neighbours in 6d -> 728

Grid sizes: 2D=6x6, 3D=6x6x6, 6D=6^6
Nodes: 2D=36, 3D=216, 6D=46656

Building graphs...

2D graph built: 0.000s, 36 nodes, 220 connections

3D graph built: 0.002s, 216 nodes, 3880 connections

6D graph built: 75.163s, 46656 nodes, 16730560 connections

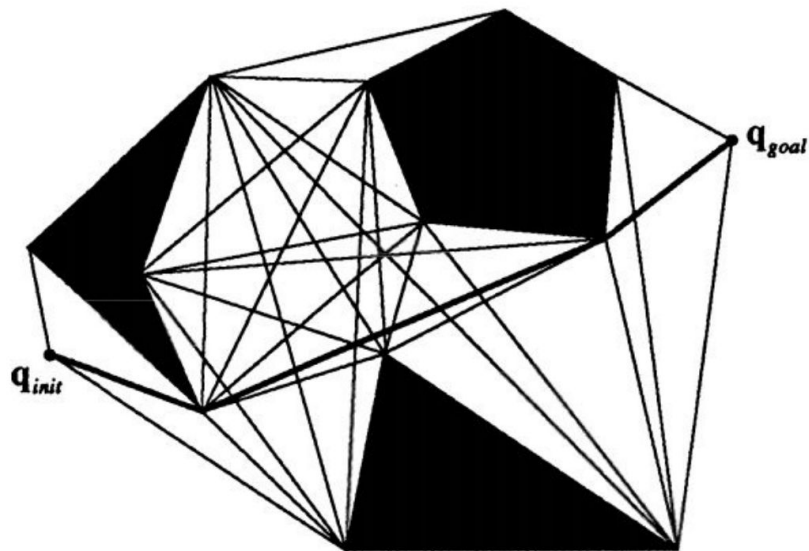
Running Dijkstra...

2D: time = 0.0001s

3D: time = 0.0009s

6D: time = 3.2846s

Visibility Graphs

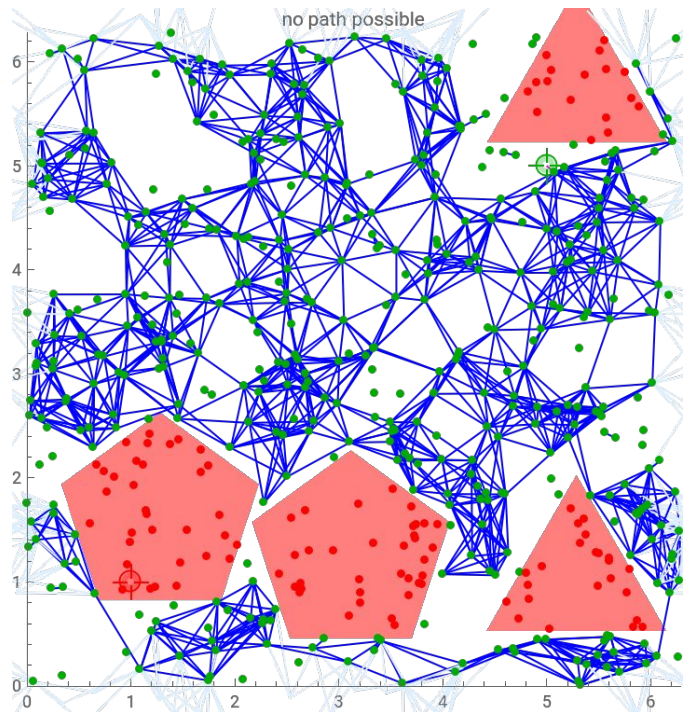


[Figure from Robot Motion Planning]

Sampling Based Planners

- Non deterministic
- Instead of mapping the whole space, keep sampling points until you find a path
- Eg. PRM, RRT

sPRM



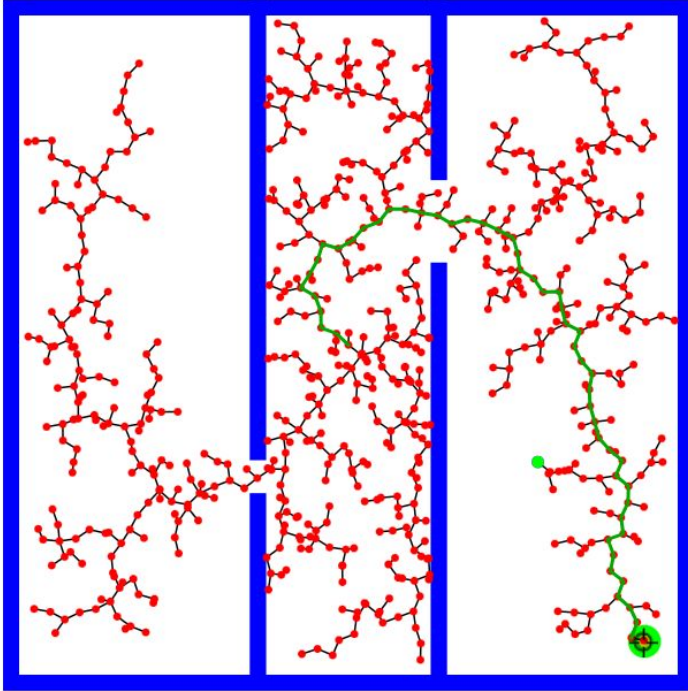
Multi Query Method

Algorithm 2: sPRM.

```
1  $V \leftarrow \{x_{\text{init}}\} \cup \{\text{SampleFree}_i\}_{i=1,\dots,n}; E \leftarrow \emptyset;$   
2 foreach  $v \in V$  do  
3    $U \leftarrow \text{Near}(G=(V,E), v, r) \setminus \{v\};$   
4   foreach  $u \in U$  do  
5     if  $\text{CollisionFree}(v,u)$  then  
6        $E \leftarrow E \cup \{(v,u), (u,v)\}$   
6 return  $G=(V,E);$ 
```

RRT

696 nodes, path length 51.



Single Query Method

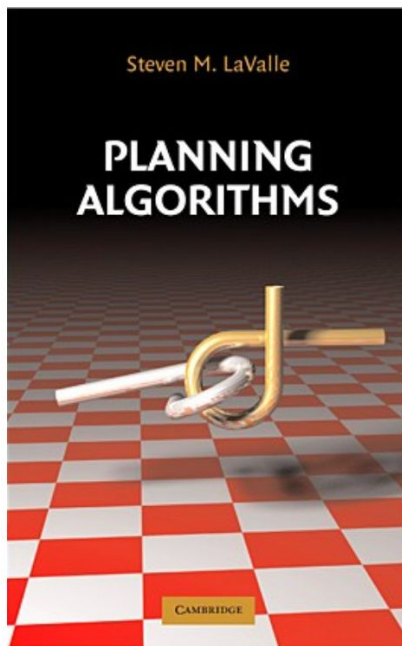
Algorithm 3: RRT.

```
1  $V \leftarrow \{x_{\text{init}}\}; E \leftarrow \emptyset;$   
2 for  $i = 1, \dots, n$  do  
3    $x_{\text{rand}} \leftarrow \text{SampleFree}_i;$   
4    $x_{\text{nearest}} \leftarrow \text{Nearest}(G = (V, E), x_{\text{rand}});$   
5    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x_{\text{rand}});$   
6   if  $\text{ObstacleFree}(x_{\text{nearest}}, x_{\text{new}})$  then  
7      $V \leftarrow V \cup \{x_{\text{new}}\}; E \leftarrow E \cup \{(x_{\text{nearest}}, x_{\text{new}})\};$   
8 return  $G = (V, E);$ 
```

There are a lot more!

[illegible]

<https://lavalle.pl/planning/>



PLANNING ALGORITHMS

By [Steven M. LaValle](#), Copyright 2006
Cambridge University Press, 842 pages
To buy: [Cambridge](#) [Amazon](#)

Cover art by [James Kuffner](#)
Art on the first page of each part by [Odri](#)

[The whole book in HTML](#)
[Courses that use the book](#)
[BibTeX file for all 1005 references](#) (edited by [Jason O'Kane](#))
[Errata](#) (First Printing)